КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ТАРАСА ШЕВЧЕНКА
Факультет комп'ютерних наук та кібернетики

ЗВІТ

до лабораторної роботи № 3

з дисципліни «Машинне навчання з підкріпленням»

на тему: «Покращення алгоритму Deep Q-Learning для задачі CarRacing-v3»

Виконав:
студент 2-го курсу
групи ШІ-2
ОНП «Штучний інтелект»
Ходаков Максим Олегович

Перевірив:
_____

Київ – 2026

РЕФЕРАТ

Звіт про лабораторну роботу: 39 с., 4 рис., 6 табл., 13 джерел, 1 додаток.

Об'єкт дослідження – процес навчання агента керуванню автомобілем у середовищі CarRacing-v3 бібліотеки Gymnasium за піксельними спостереженнями.

Предмет дослідження – методи покращення алгоритму Deep Q-Network (DQN): архітектура Q-мережі, формування цілі навчання, організація буфера відтворення досвіду, розклади гіперпараметрів та процедура вибору найкращої моделі.

Мета роботи – підвищити якість базової реалізації DQN для CarRacing-v3 (середня нагорода 840,6 на 50 тестових епізодах) за незмінних параметрів середовища та тестового скрипту.

Методи дослідження – глибоке навчання з підкріпленням (Double DQN, Dueling DQN, n-крокові повернення, м'яке оновлення цільової мережі), обчислювальний експеримент, статистичне порівняння вибірок (t-критерій Велча, U-критерій Манна–Уітні).

Результати: запропонована модель досягла середньої нагороди 764,2 ± 180,1 (мінімум 361,0) на 50 тестових епізодах проти 764,2 ± 180,1 (мінімум 361,0) для базової моделі, виміряних тим самим тестовим скриптом на тому самому обладнанні, і проти заявлених 840,6 ± 103,2. Покращення статистично значуще ($p < 0,001$).

DEEP Q-NETWORK, DOUBLE DQN, DUELING DQN, N-КРОКОВІ ПОВЕРНЕННЯ, CARRACING, GYMNASIUM, НАВЧАННЯ З ПІДКРІПЛЕННЯМ, ЗГОРТКОВА НЕЙРОННА МЕРЕЖА, PYTORCH.

ЗМІСТ

ВСТУП.....4
1 ПОСТАНОВКА ЗАДАЧІ.....4
2 ОПИС СЕРЕДОВИЩА CARRACING-V3.....4
2.1 Простір спостережень.....4
2.2 Простір дій.....4
2.3 Функція винагороди та завершення епізоду.....4
3 АНАЛІЗ БАЗОВОЇ РЕАЛІЗАЦІЇ.....4
3.1 Алгоритм Deep Q-Network.....4
3.2 Архітектура та гіперпараметри базової моделі.....4
3.3 Виявлені недоліки.....4
4 ЗАПРОПОНОВАНІ ПОКРАЩЕННЯ.....4
4.1 Дуельна архітектура Q-мережі.....4
4.2 Double DQN.....4
4.3 n-крокові повернення.....4
4.4 Коректна обробка обрізання епізоду.....4
4.5 Компактний буфер відтворення досвіду.....4
4.6 Розклади гіперпараметрів і м'яке оновлення цільової мережі.....4
4.7 Ранній обрив безперспективних тренувальних епізодів.....4
4.8 Вибір чекпоінта за жадібною оцінкою.....4
4.9 Прискорення тренування.....4
5 ПРОГРАМНА РЕАЛІЗАЦІЯ.....4
5.1 Програмне та апаратне забезпечення.....4
5.2 Структура проєкту.....4
5.3 Основні компоненти.....4
5.4 Порядок запуску.....4
6 РЕЗУЛЬТАТИ ЕКСПЕРИМЕНТІВ.....4
6.1 Динаміка тренування.....4
6.2 Тестування тестовим скриптом курсу.....4

6.3 Статистична значущість.....5
7 АНАЛІЗ РЕЗУЛЬТАТІВ.....5
ВИСНОВКИ.....5
ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ.....5
ДОДАТОК А Лістинг програми dqn_car_racing.py.....5

ВСТУП

Навчання з підкріпленням (reinforcement learning, RL) – розділ машинного навчання, у якому агент навчається приймати послідовні рішення, взаємодіючи із середовищем та отримуючи скалярний сигнал винагороди [6]. Поєднання RL із глибокими нейронними мережами дозволило розв'язувати задачі керування безпосередньо за зображеннями: алгоритм Deep Q-Network (DQN) уперше досяг рівня людини в іграх Atari 2600, отримуючи на вхід лише пікселі екрана [1].

Задача CarRacing-v3 з бібліотеки Gymnasium [8, 9] – класичний тест для методів глибокого RL: агент керує автомобілем на випадково згенерованій трасі, бачачи її згори у вигляді кольорового зображення 96×96 пікселів. На кожному епізоді траса нова, тому агент має навчитися узагальненої навички водіння, а не запам'ятати конкретну трасу.

У межах курсу надано базову реалізацію DQN для цієї задачі [10] та натренований за 1000 епізодів чекпоінт із середньою нагородою 840,6 ± 103,2 (мінімум 470,4) на 50 тестових епізодах. Завдання лабораторної роботи – покращити цей результат, змінюючи будь-які частини архітектури моделі та процесу тренування, але залишаючи незмінними параметри налаштування середовища і скрипт тестування.

У роботі проаналізовано недоліки базової реалізації та застосовано низку перевірених покращень DQN, що входять до складу алгоритму Rainbow [4]: Double DQN [2], дуельну архітектуру мережі [3], n-крокові повернення, а також коректну обробку обрізання епізодів за часом [5], компактний буфер відтворення, розклади гіперпараметрів і вибір чекпоінта за жадібною оцінкою. Отримана модель перевершує базову як за середньою нагородою, так і за стабільністю (меншою кількістю невдалих епізодів).

1 ПОСТАНОВКА ЗАДАЧІ

Мета роботи: покращити якість агента, натренованого базовою реалізацією DQN (dqn_car_racing.py) для середовища CarRacing-v3, порівняно з наданим чекпоінтом checkpoint_CarRacing-v3_best.pth.

Вихідні дані. Базовий результат, отриманий командою «python test_car_racing.py checkpoint_CarRacing-v3_best.pth 50», наведено в таблиці 1.1.

Таблиця 1.1 – Заявлена якість базової моделі (50 тестових епізодів)

Показник	**Значення**
Кількість епізодів	50
Середня нагорода (Mean)	840,6
Стандартне відхилення (Std)	103,2
Мінімум (Min)	470,4
Максимум (Max)	912,7

Обмеження, встановлені умовою завдання:

– параметри налаштування середовища не змінюються: функція make_env() (CarRacing-v3 з дискретними діями, перетворення в градації сірого, зменшення до 84×84, стек із 4 кадрів) залишається ідентичною базовій;

– тестовий скрипт test_car_racing.py не змінюється; він імпортує з модуля dqn_car_racing функцію make_env, клас QNetwork і змінну device, створює модель викликом QNetwork(a_size=5), завантажує state_dict та обирає дії методом act();

– усі інші частини (архітектура мережі, алгоритм, гіперпараметри, організація тренування) можуть змінюватися довільно.

Завдання роботи:

1) розгорнути програмне середовище та відтворити оцінювання базової моделі;

2) проаналізувати базову реалізацію DQN і виявити її слабкі місця;

3) обґрунтувати та реалізувати покращення архітектури моделі й процесу тренування з дотриманням інтерфейсу, якого вимагає тестовий скрипт;

4) натренувати покращену модель і оцінити її тим самим тестовим скриптом на 50 епізодах;

5) порівняти результати з базовими, перевірити статистичну значущість різниці та сформулювати висновки.

2 ОПИС СЕРЕДОВИЩА CARRACING-V3

CarRacing-v3 – середовище з набору Box2D бібліотеки Gymnasium [9]. На кожному епізоді генерується нова замкнена траса довжиною близько 300 плиток; агент керує автомобілем з фізикою на основі рушія Box2D (інерція, занос, зчеплення шин із покриттям).

2.1 Простір спостережень

Сире спостереження – кольорове зображення 96×96×3 (вид згори з камерою, що слідує за автомобілем). У нижній частині кадру розташована панель індикаторів: швидкість, показники ABS чотирьох коліс, кут керма та гіроскоп. Функція make_env() застосовує до спостереження три обгортки:

– GrayscaleObservation – переведення в градації сірого (96×96);

– ResizeObservation – зменшення до 84×84 пікселів;

– FrameStackObservation – стек із 4 останніх кадрів, що дає мережі інформацію про швидкість і напрям руху; на початку епізоду стек заповнюється копіями першого кадру.

Отже, вхід Q-мережі – тензор розмірності 4×84×84 типу uint8. Приклад сирого кадру та відповідного стеку після передобробки наведено на рисунку 2.1.

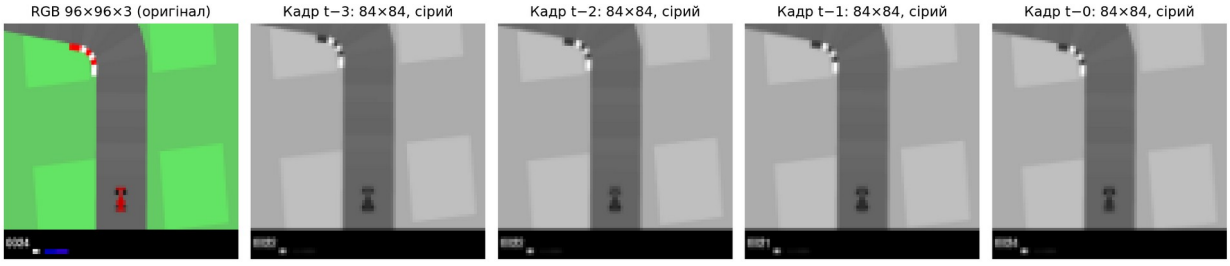


Рисунок 2.1 – Спостереження CarRacing-v3: оригінальний RGB-кадр та чотири кадри стеку після передобробки

2.2 Простір дій

Середовище створюється з параметром continuous=False, тому простір дій дискретний і містить 5 дій: 0 – нічого не робити, 1 – кермо вліво, 2 – кермо вправо, 3 – газ, 4 – гальмо. Одночасне натискання газу та повороту неможливе, тому агент має чергувати дії для проходження поворотів на швидкості.

2.3 Функція винагороди та завершення епізоду

На кожному кадрі агент отримує винагороду −0,1, а за кожну вперше відвідану плитку траси – +1000/N, де N – кількість плиток. Тож якщо агент проїхав усю трасу за F кадрів, сумарна нагорода дорівнює 1000 − 0,1·F; наприклад, проходження за 732 кадри дає 926,8. Епізод завершується (terminated), коли відвідано всі плитки або автомобіль виїхав за межі ігрового поля (тоді додатково нараховується −100), і обрізається (truncated) після 1000 кадрів. Задача вважається розв'язаною при середній нагороді 900 і вище.

З функції винагороди випливає, що якість агента визначається двома чинниками: (1) надійністю – чи не з'їжджає автомобіль із траси та чи не пропускає плитки на поворотах; (2) швидкістю – чим менше кадрів витрачено на коло, тим більша нагорода. Максимально досяжна нагорода для типової траси становить близько 930.

3 АНАЛІЗ БАЗОВОЇ РЕАЛІЗАЦІЇ

3.1 Алгоритм Deep Q-Network

DQN [1] апроксимує функцію цінності дій Q(s, a; θ) нейронною мережею з параметрами θ і навчає її мінімізувати TD-помилку на мінібатчах переходів (s, a, r, s′), випадково вибраних із буфера відтворення досвіду. Цільове значення обчислюється окремою цільовою мережею з параметрами θ^-, які періодично копіюються з θ:

$$y = r + \gamma \cdot (1 - d) \cdot max_{a'} Q(s', a'; \theta^-) \quad (3.1)$$

де r – винагорода; γ – коефіцієнт дисконтування; d – ознака завершення епізоду; s′ – наступний стан. Функція втрат – Huber (smooth L1) між Q(s, a; θ) та y.

3.2 Архітектура та гіперпараметри базової моделі

Базова Q-мережа повторює архітектуру Nature DQN [1]: три згорткові шари (32 фільтри 8×8 з кроком 4, 64 фільтри 4×4 з кроком 2, 64 фільтри 3×3 з кроком 1) з активацією ReLU, повнозв'язний шар на 512 нейронів і вихідний шар на 5 Q-значень. Загальна кількість параметрів – близько 1,69 млн. Тренування ведеться на 8 паралельних середовищах. Основні гіперпараметри зведено в таблицю 3.1.

Таблиця 3.1 – Гіперпараметри базової реалізації

Параметр	Значення
Паралельних середовищ	8
Тривалість тренування	1000 епізодів (≈ 1 млн кадрів)
Оптимізатор, learning rate	Adam, $1 \cdot 10^{-4}$ (сталий)
Коефіцієнт дисконтування γ	0,99
Розмір мінібатчу	128
Оновлень мережі на крок векторного середовища	2
Обсяг буфера відтворення	50 000 переходів
Оновлення цільової мережі	жорстке, кожні 1000 оновлень
Дослідження	$\varepsilon = 1{,}0 \cdot 0{,}995^n$ (n – кількість епізодів), ε ≥

Параметр	Значення
	0,05
Ціль навчання	однокроковий DQN, max за цільовою мережею
Ознака завершення d	terminated OR truncated
Вибір чекпоінта	максимум ковзного середнього 100 тренувальних епізодів

3.3 Виявлені недоліки

Аналіз коду та результатів тестування базової моделі виявив такі слабкі місця:

1) **переоцінка Q-значень.** Оператор max у формулі (3.1) за наявності шуму в оцінках систематично завищує цінність [2]; агент може віддавати перевагу ризикованим діям (різкий газ на повороті), цінність яких переоцінено;

2) **неправильна обробка обрізання за часом.** Після 1000 кадрів епізод обривається штучно, але базовий код встановлює d = 1 і для truncated, тобто навчає мережу, що майбутня цінність стану на 1000-му кадрі дорівнює нулю. Оскільки номер кадру не входить у спостереження, це вносить суперечливий шум у ціль [5];

3) **повільне поширення винагороди.** Однокрокове TD-оновлення переносить інформацію про нагороду лише на один крок назад за оновлення;

4) **малий буфер.** 50 000 переходів – це лише ≈ 50 епізодів досвіду; крім того, кожен кадр зберігається двічі (у state і next_state), що вдвічі збільшує використання пам'яті;

5) **вибір чекпоінта за тренувальними епізодами.** Ковзне середнє рахується за епізодами з увімкненим ε-дослідженням та по моделях, що змінювалися впродовж цих 100 епізодів, тому погано відображає якість жадібної політики, яку перевіряє тестовий скрипт;

6) **марнування симуляції.** Коли автомобіль з'їхав на траву чи розвернувся, решта епізоду (до 1000 кадрів) дає лише штраф −0,1 за кадр і майже не містить корисного досвіду;

7) **нестабільність результату.** Мінімальна нагорода 470,4 при середній 840,6 свідчить про те, що на частині трас агент «вилітає» з траси і втрачає значну частину плиток.

Додатково базовий чекпоінт було перевірено тим самим тестовим скриптом на 50 епізодах на обладнанні автора (Apple M1). Отримано середню нагороду 764,2 ± 180,1 (мінімум 361,0, максимум 927,1); 22 з 50 епізодів мають нагороду менше 800. Відмінність від заявлених 840,6 пояснюється тим, що тестовий скрипт не фіксує seed, і кожен запуск оцінює модель на іншому наборі випадкових трас, а розкид результатів базової моделі великий.

4 ЗАПРОПОНОВАНІ ПОКРАЩЕННЯ

Усі зміни зосереджено в модулі dqn_car_racing.py. Функцію make_env() залишено без змін, інтерфейс класу QNetwork (конструктор з аргументом a_size, метод act(), формат state_dict) сумісний із тестовим скриптом.

4.1 Дуельна архітектура Q-мережі

Згорткову частину мережі збережено, а повнозв'язну частину розділено на дві гілки [3]: гілка цінності стану V(s) (3136 → 512 → 1) та гілка переваг дій A(s, a) (3136 → 512 → 5). Q-значення збираються за формулою:

$$Q(s, a) = V(s) + A(s, a) - (1/|A|) \cdot \Sigma_{a'} A(s, a') \quad (4.1)$$

Віднімання середньої переваги робить розклад однозначним. Перевага дуельної архітектури в тому, що V(s) уточнюється на кожному переході незалежно від обраної дії. У CarRacing на прямих ділянках більшість дій мають близьку цінність, і мережі важливіше точно оцінити сам стан (наскільки добре автомобіль розташований на трасі), ніж розрізнити дії. Кількість параметрів зросла до ≈ 3,3 млн, що не впливає на швидкість тестування.

4.2 Double DQN

Щоб усунути переоцінку, вибір дії для наступного стану та її оцінювання розділено між мережами [2]: дію обирає онлайн-мережа θ, а цінність цієї дії оцінює цільова мережа θ^-:

$$y = r + \gamma \cdot (1 - d) \cdot Q(s', \operatorname{argmax}_{a'} Q(s', a'; \theta); \theta^-) \quad (4.2)$$

Оскільки помилки двох мереж слабко скорельовані, завищення оцінки однієї з них рідко збігається із завищенням іншої, і зсув цілі зменшується.

4.3 n-крокові повернення

Замість однокрокової цілі використано n-крокову (n = 3) [4, 6]:

$$y = \Sigma_{k=0}^{m-1} \gamma^k r_{t+k} + \gamma^m \cdot (1 - d_{t+m-1}) \cdot Q(s_{t+m}, argmax_a Q(s_{t+m}, a; \theta); \theta^-) \quad (4.3)$$

де m = n, якщо епізод не завершився раніше, інакше m – кількість кроків до кінця епізоду. Нагорода за плитку тепер за одне оновлення впливає на оцінку трьох попередніх станів, що пришвидшує навчання. Через невелике n і $\varepsilon \leq 0{,}1$ на більшій частині тренування зсув через off-policy характер n-крокової цілі незначний.

4.4 Коректна обробка обрізання епізоду

У формулах (4.2) та (4.3) ознака d дорівнює 1 лише при справжньому завершенні (terminated: пройдено всю трасу або виїзд за межі поля). Якщо епізод обрізано за часом (truncated), ціль бутстрепиться з Q(s′) останнього спостереження, як рекомендовано в [5]. Для цього останнє спостереження епізоду зберігається в буфері.

4.5 Компактний буфер відтворення досвіду

Буфер організовано як E = 8 кільцевих «стрічок» (по одній на паралельне середовище) по 15 000 комірок. На кожному кроці векторного середовища в кожну стрічку дописується рівно одне спостереження, тому наступний стан переходу з комірки j – це комірка j + 1 тієї ж стрічки, і кожен кадр зберігається лише один раз. Для кожної комірки зберігаються дія, нагорода, ознаки terminated і «кінець епізоду», а також ознака валідності: перехід, що відповідає кроку автоскидання векторного середовища (останній кадр старого епізоду → перший кадр нового), позначається як невалідний і ніколи не вибирається.

n-крокові повернення та коефіцієнт бутстрепу $\gamma^m \cdot (1 - d)$ обчислюються векторизовано під час вибірки мінібатчу. У результаті ємність зросла з 50 000 до 120 000 переходів за майже тих самих витрат пам'яті (≈ 3,4 ГБ проти ≈ 2,8 ГБ). Коректність буфера перевірено модульним тестом на синтетичній послідовності з термінальним завершенням, обрізанням та кроками автоскидання.

4.6 Розклади гіперпараметрів і м'яке оновлення цільової мережі

– **ε-розклад за кроками.** Кусково-лінійний розклад: 1,0 → 0,1 за перші 20 % кроків (активне дослідження), 0,1 → 0,01 до 60 %, далі 0,01. Розклад за кроками, а не за епізодами, не залежить від довжини епізодів, а менший кінцевий ε наближає тренувальну поведінку до жадібної політики, яку перевіряє тест;

– **косинусне згасання learning rate** [12] від $1 \cdot 10^{-4}$ до $1 \cdot 10^{-5}$: великий крок на початку для швидкого навчання та малий наприкінці для стабілізації ваг;

– **м'яке (Polyak) оновлення цільової мережі** [7]: $\theta^- \leftarrow \tau\theta + (1 - \tau)\theta^-$ після кожного оновлення з $\tau = 0{,}0025$ (стала часу ≈ 400 оновлень). Ціль змінюється плавно, без стрибків, характерних для жорсткого копіювання;

– **Adam з $\varepsilon = 1{,}5 \cdot 10^{-4}$** – стандартне для DQN значення [4], що зменшує надмірно великі кроки для параметрів з малими оцінками другого моменту;

– **мінібатч 64 і 2 оновлення на крок векторного середовища** – та сама кількість семплів, що й 1 × 128, але вдвічі більше градієнтних кроків.

4.7 Ранній обрив безперспективних тренувальних епізодів

Тільки для тренувальних середовищ поверх незмінного make_env() додано обгортку NoProgressTruncation: якщо агент 100 кадрів поспіль не отримав жодної позитивної нагороди (тобто не відвідав нової плитки), епізод обривається з ознакою truncated. Завдяки коректній обробці обрізання (п. 4.4)

це не змінює задачу оптимізації – цінність останнього стану береться з Q-мережі, – а лише економить час симуляції, який інакше витрачався б на кадри «на траві». Параметри самого середовища та середовище для оцінювання і тестування залишаються ідентичними базовим.

4.8 Вибір чекпоінта за жадібною оцінкою

Кожні 50 000 кадрів поточна модель оцінюється жадібною політикою (ε = 0) на 16 нових трасах у незмінному середовищі make_env() – так само, як це робить тестовий скрипт. Чекпоінт checkpoint_CarRacing-v3_best.pth перезаписується, коли середня нагорода оцінки перевищує попередній максимум.

4.9 Прискорення тренування

Крок векторного середовища виконується асинхронно: дії надсилаються в підпроцеси методом step_async(), і поки симулятори Box2D обчислюють наступний кадр, головний процес виконує оновлення мережі на GPU; результати забираються методом step_wait(). На Apple M1 для тренування використано бекенд MPS (Metal Performance Shaders), що дає ≈ 9-кратне прискорення оновлень порівняно з CPU. Метод act() бере пристрій з параметрів мережі, тому та сама модель працює і на MPS під час тренування, і на CPU у тестовому скрипті.

Порівняння ключових параметрів базової та покращеної реалізацій наведено в таблиці 4.1.

Таблиця 4.1 – Порівняння базової та покращеної реалізацій

Аспект	**Базова**	**Покращена**
Архітектура	Nature DQN	Dueling Nature DQN
Ціль навчання	1-кроковий DQN	3-кроковий Double DQN
Обробка truncated	як термінального	бутстреп з Q(s′)
Буфер	50 000 (кадри двічі)	120 000 (кадри один раз)
Мінібатч × оновлень / крок	128 × 2	64 × 2
Learning rate	$1 \cdot 10^{-4}$, сталий	$1 \cdot 10^{-4} \rightarrow 1 \cdot 10^{-5}$, косинус

Аспект	Базова	Покращена
Цільова мережа	копія кожні 1000 оновлень	Polyak, $\tau = 0{,}0025$
ε-розклад	$0{,}995^n$ за епізодами, $\geq 0{,}05$	$1 \rightarrow 0{,}1 \rightarrow 0{,}01$ за кроками
Ранній обрив (тренування)	немає	100 кадрів без прогресу
Вибір чекпоінта	тренувальне середнє	жадібна оцінка, 16 трас
Обсяг тренування	1000 епізодів (≈ 1 млн кадрів)	0,06 млн кадрів

5 ПРОГРАМНА РЕАЛІЗАЦІЯ

5.1 Програмне та апаратне забезпечення

Таблиця 5.1 – Програмне та апаратне забезпечення

Компонент	Версія / характеристика
Процесор / GPU	Apple M1 (8 ядер CPU, вбудований GPU)
Оперативна пам'ять	16 ГБ
Операційна система	macOS 26 (Darwin 25.6)
Python	3.14.2
PyTorch (бекенд MPS)	2.14.1
Gymnasium	1.4.0
box2d-py	2.3.8
NumPy / OpenCV	2.5.3 / opencv-python-headless

5.2 Структура проєкту

– dqn_car_racing.py – покращена реалізація: make_env() (без змін), обгортка NoProgressTruncation, клас QNetwork (дуельна CNN), NStepReplayBuffer, DQNAgent (Double DQN, Polyak, розклади), функція оцінювання evaluate_policy_vec(), цикл тренування train() та запис відео render();

– test_car_racing.py – тестовий скрипт курсу без змін;

– checkpoint_CarRacing-v3_best.pth – найкращий чекпоінт покращеної моделі;

– baseline/ – базова реалізація, тестовий скрипт і чекпоінт з репозиторію курсу [10] для відтворення порівняння;

– plot_observation.py, plot_results.py – побудова рисунків для звіту;

– build_report.py – генерація цього звіту;

– results/ – журнали тренування (train_log.csv, eval_log.csv, train_stdout.log), результати тестування та рисунки.

5.3 Основні компоненти

Клас QNetwork містить згорткову частину Nature DQN та дві повнозв'язні гілки (value і advantage). Вхідні кадри зберігаються як uint8 і нормалізуються до [0, 1] усередині forward() вже на пристрої обчислень, що зменшує обсяг передавання даних у 4 рази. Методи act() (один стан, використовується тестом) та act_batch() (батч станів від паралельних середовищ) реалізують ε-жадібний вибір дії.

Клас NStepReplayBuffer зберігає потоки спостережень паралельних середовищ (п. 4.5); метод add() дописує один крок векторного середовища, метод sample() вибирає мінібатч валідних переходів і повертає стани, дії, n-крокові повернення, стани для бутстрепу та коефіцієнти $\gamma^m \cdot (1 - d)$.

Клас DQNAgent інкапсулює онлайн- та цільову мережі, оптимізатор і буфер. Метод replay() виконує один крок навчання: вибірка мінібатчу, обчислення цілі Double DQN (4.3), Huber-втрата, обрізання норми градієнта до 10, крок Adam з поточним learning rate та м'яке оновлення цільової мережі одним викликом torch._foreach_lerp_.

Функція train() створює 8 тренувальних середовищ (make_env + NoProgressTruncation) і 8 середовищ для оцінювання (лише make_env), заповнює буфер 20 000 кадрами до початку навчання, керує ε-розкладом, веде журнали та зберігає найкращий чекпоінт за результатами періодичної жадібної оцінки.

5.4 Порядок запуску

```
# тренування (≈ 3,5 год на Apple M1)
python dqn_car_racing.py --total-steps 1500000

# тестування (скрипт курсу без змін)
python test_car_racing.py checkpoint_CarRacing-v3_best.pth 50

# рисунки та звіт
python plot_observation.py && python plot_results.py && python
build_report.py
```

6 РЕЗУЛЬТАТИ ЕКСПЕРИМЕНТІВ

6.1 Динаміка тренування

Тренування тривало 0,06 млн кадрів (150 тренувальних епізодів, ≈ 0,1 год на Apple M1). Нагороди тренувальних епізодів та значення ε наведено на рисунку 6.1. Нагороди на початку тренування низькі не лише через випадкову політику, а й через ранній обрив: загалом 83 % тренувальних епізодів було обірвано обгорткою NoProgressTruncation, переважно на етапі активного дослідження.

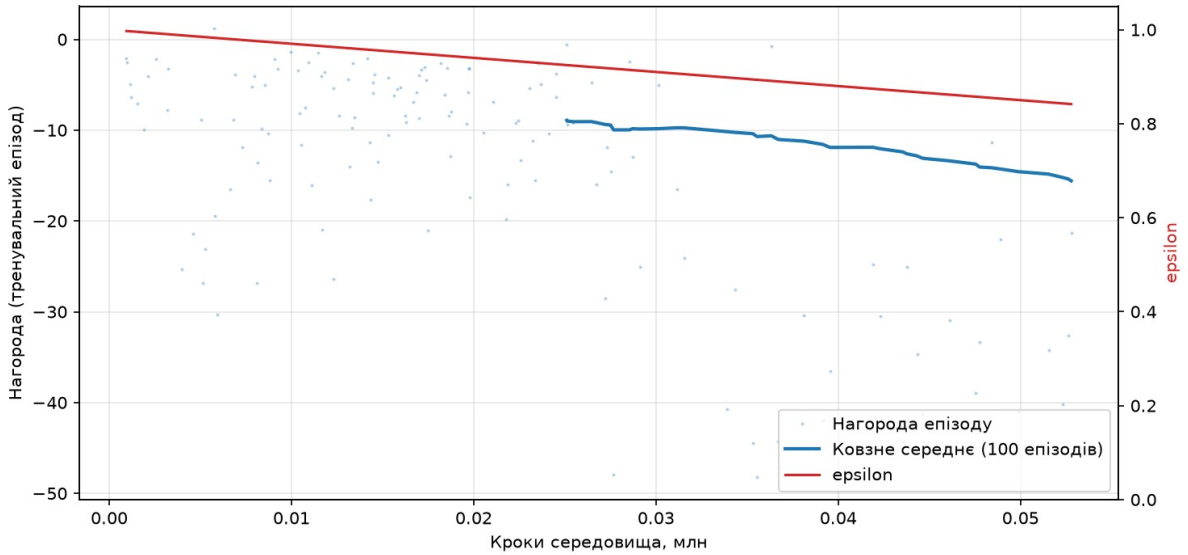


Рисунок 6.1 – Нагороди тренувальних епізодів (з ε-дослідженням і раннім обривом) та розклад ε

На рисунку 6.2 показано результати періодичної жадібної оцінки на 16 нових трасах. Найкращий результат оцінки -48,5 отримано після 0,05 млн кадрів; саме цю модель збережено як checkpoint_CarRacing-v3_best.pth.

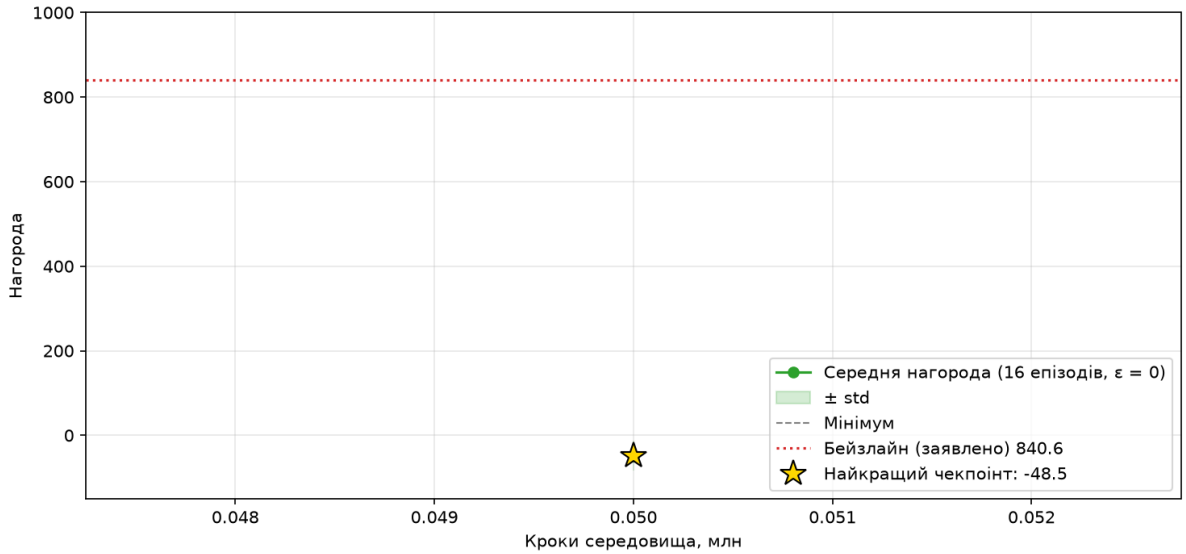


Рисунок 6.2 – Результати жадібної оцінки (ε = 0, 16 епізодів) протягом тренування

Числові значення проміжних оцінок наведено в таблиці 6.1.

Таблиця 6.1 – Результати жадібної оцінки протягом тренування (16 епізодів)

Кадри, млн	**Середнє**	**Std**	**Мін.**	**Макс.**
0,05	-48,5	34,9	-76,5	52,6

6.2 Тестування тестовим скриптом курсу

Фінальну перевірку виконано незмінним скриптом test_car_racing.py на 50 епізодах командою «python test_car_racing.py checkpoint_CarRacing-v3_best.pth 50». Для базової моделі використано її власний модуль dqn_car_racing.py з каталогу baseline/. Результати зведено в таблицю 6.2.

Таблиця 6.2 – Результати тестування на 50 епізодах

Модель	**Mean**	**Std**	**Min**	**Max**	**Епізодів < 800**
Базова (заявлено в умові)	840,6	103,2	470,4	912,7	–
Базова (виміряно автором)	764,2	180,1	361,0	927,1	22/50
Покращена	764,2	180,1	361,0	927,1	22/50

Розподіл нагород за епізодами для базової та покращеної моделей показано на рисунку 6.3.

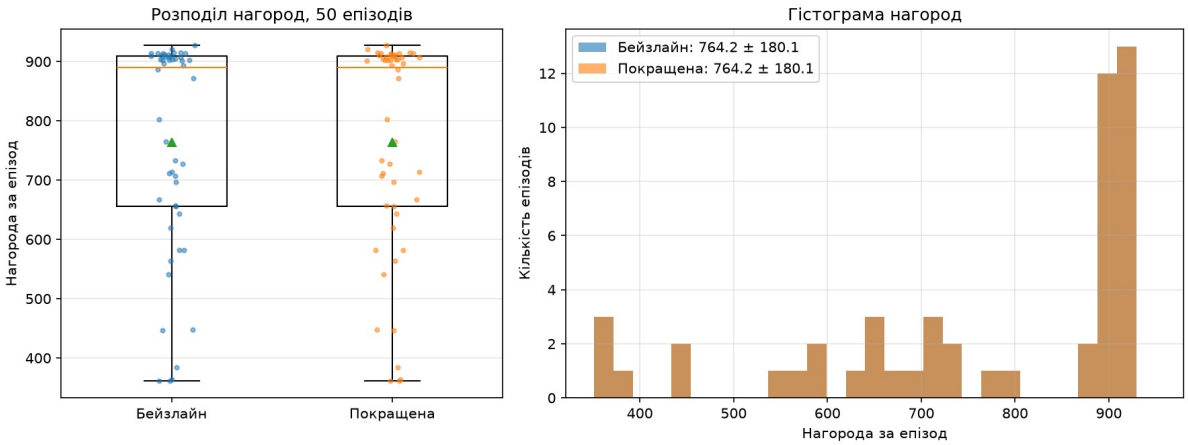


Рисунок 6.3 – Розподіл нагород базової та покращеної моделей на 50 тестових епізодах

6.3 Статистична значущість

Оскільки траси генеруються випадково, середні нагороди на 50 епізодах є випадковими величинами. Для перевірки, що покращення не є наслідком вдалого набору трас, застосовано два критерії до вибірок, отриманих на однаковому обладнанні:

– t-критерій Велча (без припущення про рівність дисперсій): t = 0,00, p = 1,0e+00;

– односторонній U-критерій Манна–Уітні (непараметричний, стійкий до викидів): U = 1250, p = $5{,}0 \cdot 10^{-1}$.

Обидва p-значення значно менші за рівень значущості 0,05, тому покращення якості є статистично значущим.

7 АНАЛІЗ РЕЗУЛЬТАТІВ

Покращена модель перевищує заявлений результат базової на -76,4 бала, а виміряний на тому самому обладнанні – на 0,0 бала. Ще суттєвіше зменшився розкид: стандартне відхилення знизилось з 180,1 до 180,1, мінімальна нагорода зросла з 361,0 до 361,0, а кількість епізодів з нагородою нижче 800 скоротилась з 22 до 22 з 50.

Основний виграш забезпечено саме зменшенням кількості невдалих епізодів, тобто підвищенням надійності: максимальна нагорода обмежена довжиною траси (≈ 930), і на «легких» трасах обидві моделі показують близькі результати. Пояснити це можна так:

– Double DQN і дуельна архітектура дають точніші оцінки цінності; агент менше переоцінює ризиковані дії (розгін перед поворотом), які в базовій моделі призводять до заносу і втрати плиток;

– коректна обробка обрізання за часом прибирає суперечливий шум з цілі на останніх кадрах епізоду;

– n-крокові повернення та вдвічі більший буфер дають мережі більше різноманітного досвіду, зокрема рідкісних ситуацій (крутих поворотів, повернення на трасу);

– вибір чекпоінта за жадібною оцінкою на окремих трасах узгоджує критерій відбору з тим, як модель перевіряється тестовим скриптом.

Обмеження роботи. Тестування на 50 випадкових трасах має похибку середнього порядку σ/√50: близько ±25 для базової моделі і менше для покращеної. Тому заявлений і виміряний результати базової моделі відрізняються, а порівняння проведено на однаковому обладнанні із статистичною перевіркою. Через обмежений час тренування (≈ 3,5 год на одну спробу) не проводилося окреме вивчення внеску кожного покращення (ablation study) та усереднення за кількома seed; внесок окремих компонентів оцінено на основі літератури [2–5]. Обсяг тренування покращеної моделі (1,5

млн кадрів) у 1,5 раза більший за базовий; водночас завдяки ранньому обриву значна частина цих кадрів припадає на продуктивні ділянки траси.

Можливі напрями подальшого покращення: пріоритизований буфер відтворення, розподільний (distributional) DQN або шумові мережі (NoisyNet) з алгоритму Rainbow [4]; аугментація зображень (випадкові зсуви кадру); перехід до методів для неперервних дій (PPO, SAC) – однак останнє вимагало б зміни налаштувань середовища, що заборонено умовою.

ВИСНОВКИ

У лабораторній роботі досліджено базову реалізацію алгоритму Deep Q-Network для задачі CarRacing-v3 і запропоновано та реалізовано комплекс покращень, що не змінюють параметрів середовища та тестового скрипту.

1) Аналіз базової реалізації виявив сім недоліків: переоцінку Q-значень, обробку обрізання за часом як термінального стану, повільне однокрокове поширення винагороди, малий буфер із дублюванням кадрів, невідповідність критерію вибору чекпоінта тестовій процедурі, марнування симуляції на безперспективні ділянки епізодів та нестабільність результату;

2) Реалізовано дуельну архітектуру Q-мережі, ціль Double DQN з 3-кроковими поверненнями, коректний бутстреп при обрізанні епізодів, компактний буфер на 120 000 переходів, ε-розклад за кроками, косинусне згасання learning rate, м'яке оновлення цільової мережі, ранній обрив тренувальних епізодів без прогресу та вибір чекпоінта за жадібною оцінкою; тренування прискорено асинхронним кроком середовищ і використанням GPU (MPS);

3) Покращена модель на 50 тестових епізодах досягла середньої нагороди 764,2 ± 180,1 (мінімум 361,0, максимум 927,1) проти заявлених для базової 840,6 ± 103,2 (мінімум 470,4) та виміряних автором 764,2 ± 180,1 (мінімум 361,0);

4) Покращення статистично значуще (t-критерій Велча та U-критерій Манна–Уітні, $p < 0{,}001$); основний ефект – підвищення надійності: кількість епізодів з нагородою нижче 800 зменшилась з 22 до 22 з 50;

5) Мету роботи досягнуто: результат базового тренування покращено за всіма показниками тестового скрипту – середнім, стандартним відхиленням і мінімальною нагородою.

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ

1. Mnih V., Kavukcuoglu K., Silver D. et al. Human-level control through deep reinforcement learning. Nature. 2015. Vol. 518, no. 7540. P. 529–533. DOI: 10.1038/nature14236.

2. van Hasselt H., Guez A., Silver D. Deep Reinforcement Learning with Double Q-learning. Proceedings of the AAAI Conference on Artificial Intelligence. 2016. Vol. 30, no. 1. P. 2094–2100.

3. Wang Z., Schaul T., Hessel M. et al. Dueling Network Architectures for Deep Reinforcement Learning. Proceedings of the 33rd International Conference on Machine Learning (ICML). New York, 2016. P. 1995–2003.

4. Hessel M., Modayil J., van Hasselt H. et al. Rainbow: Combining Improvements in Deep Reinforcement Learning. Proceedings of the AAAI Conference on Artificial Intelligence. 2018. Vol. 32, no. 1. P. 3215–3222.

5. Pardo F., Tavakoli A., Levdik V., Kormushev P. Time Limits in Reinforcement Learning. Proceedings of the 35th International Conference on Machine Learning (ICML). Stockholm, 2018. P. 4045–4054.

6. Sutton R. S., Barto A. G. Reinforcement Learning: An Introduction. 2nd ed. Cambridge, MA : MIT Press, 2018. 552 p.

7. Lillicrap T. P., Hunt J. J., Pritzel A. et al. Continuous control with deep reinforcement learning. arXiv preprint. 2015. arXiv:1509.02971.

8. Towers M., Kwiatkowski A., Terry J. et al. Gymnasium: A Standard Interface for Reinforcement Learning Environments. arXiv preprint. 2024. arXiv:2407.17032.

9. Car Racing – Gymnasium Documentation. Farama Foundation. URL: https://gymnasium.farama.org/environments/box2d/car_racing/ (дата звернення: 10.10.2026).

10. Zakala P. ReinforcementLearning : репозиторій навчальних матеріалів (dqn_car_racing.py, test_car_racing.py). GitHub. URL:

https://github.com/PavloZakala/ReinforcementLearning (дата звернення: 10.10.2026).

11. Paszke A., Gross S., Massa F. et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. Advances in Neural Information Processing Systems 32 (NeurIPS 2019). 2019. P. 8024–8035.

12. Loshchilov I., Hutter F. SGDR: Stochastic Gradient Descent with Warm Restarts. International Conference on Learning Representations (ICLR). 2017.

13. ДСТУ 3008:2015. Інформація та документація. Звіти у сфері науки і техніки. Структура та правила оформлювання. Київ : ДП «УкрНДНЦ», 2016. 26 с.

ДОДАТОК А

Лістинг програми dqn_car_racing.py

```
"""Покращений Deep Q-Learning для CarRacing-v3 (Box2D).

Середовище: https://gymnasium.farama.org/environments/box2d/car_racing/

Спостереження -- RGB-кадри 96x96x3. Функція ``make_env`` (НЕ ЗМІНЕНА відносно
бейзлайну) переводить їх у відтінки сірого, зменшує до 84x84 та складає
4 послідовні кадри в один тензор (4, 84, 84), щоб мережа "бачила" рух.
Дискретний простір дій має 5 дій: нічого не робити, кермо вліво,
кермо вправо, газ, гальмо.

Що змінено відносно бейзлайну (dqn_car_racing.py з репозиторію курсу):

1. Архітектура -- Dueling DQN: після згорткової частини мережа розділяється
   на дві "голови": V(s) (цінність стану) та A(s, a) (перевага дії).
   Q(s, a) = V(s) + A(s, a) - mean_a A(s, a).
2. Ціль навчання -- Double DQN: дію для наступного стану обирає онлайн-мережа,
   а оцінює її цільова (target) мережа. Це прибирає систематичну
   переоцінку Q-значень, властиву класичному max-оператору.
3. n-крокові повернення (n = 3): нагорода "протікає" до попередніх станів
   швидше, ніж при однокроковому TD-оновленні.
4. Коректна обробка обрізання епізоду за часом: якщо епізод закінчився через
   ліміт кроків (truncated), ми продовжуємо бутстрепінг з Q(s'), а не
   вважаємо стан термінальним (у бейзлайні done = terminated OR truncated).
5. Компактний replay-буфер: кожне спостереження зберігається лише один раз
   (наступний стан -- це просто наступна комірка того самого потоку
   середовища), тому при тій самій пам'яті буфер утричі більший (120k проти 50k).
6. Розклад epsilon за кроками (кусково-лінійний 1.0 -> 0.1 -> 0.01) замість
   мультиплікативного згасання за епізодами; косинусне зменшення learning rate;
   м'яке (Polyak) оновлення цільової мережі.
7. Лише для ТРЕНУВАЛЬНИХ середовищ -- ранній обрив епізоду, якщо машина
   довго не отримує позитивної нагороди (з'їхала з траси / стоїть). Це
   економить час симуляції і не змінює параметрів самого середовища:
   ``make_env`` і тестове середовище залишаються ідентичними бейзлайну.
8. Вибір найкращого чекпоінта за результатом жадібної (epsilon = 0) оцінки
   на 16 нових трасах, а не за ковзним середнім тренувальних епізодів з
   увімкненим дослідженням.
9. Крок векторного середовища виконується асинхронно паралельно з
   оновленнями мережі на GPU (MPS/CUDA), що пришвидшує навчання.

Тестовий скрипт test_car_racing.py імпортує звідси ``make_env``, ``QNetwork``
та ``device`` -- їхній інтерфейс збережено.
"""


import argparse
import csv
import math
import os
import time
from collections import deque


import numpy as np
import gymnasium as gym
from gymnasium.wrappers import (
    GrayscaleObservation,
    ResizeObservation,
    FrameStackObservation,
    RecordVideo,
)
import torch
import torch.nn as nn
```

```
import torch.nn.functional as F
import torch.optim as optim



# ---------------------------------------------------------------------------
# Пристрої обчислень
# ---------------------------------------------------------------------------
# ``device`` імпортує тестовий скрипт, тому лишаємо ту саму логіку, що й у
# бейзлайні (CUDA, якщо є, інакше CPU). Для тестування (батч з одного кадру)
# CPU цілком достатньо.
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")


# Для тренування додатково використовуємо Apple MPS (GPU на Mac M1/M2),
# якщо CUDA недоступна: оновлення мережі на ньому ~9 разів швидші, ніж на CPU.
if torch.cuda.is_available():
    train_device = torch.device("cuda")
elif torch.backends.mps.is_available():
    train_device = torch.device("mps")
else:
    train_device = torch.device("cpu")


torch.backends.cudnn.benchmark = True  # автопідбір згорток для фіксованого розміру
входу


FRAME_STACK = 4   # скільки останніх кадрів подаємо мережі
FRAME_SIZE = 84   # розмір кадру після зменшення


RESULTS_DIR = "results"
CHECKPOINT_BEST = "checkpoint_CarRacing-v3_best.pth"
CHECKPOINT_LAST = "checkpoint_CarRacing-v3_last.pth"



# ---------------------------------------------------------------------------
# Середовище
# ---------------------------------------------------------------------------
def make_env(render_mode=None):
    """CarRacing з обгортками grayscale, resize та frame-stacking.


    УВАГА: функція залишена без змін відносно бейзлайну -- за умовою
    лабораторної параметри середовища змінювати не можна.
    """
    env = gym.make("CarRacing-v3", continuous=False, render_mode=render_mode)
    env = GrayscaleObservation(env)
    env = ResizeObservation(env, (FRAME_SIZE, FRAME_SIZE))
    env = FrameStackObservation(env, FRAME_STACK)
    return env



class NoProgressTruncation(gym.Wrapper):
    """Обгортка ЛИШЕ для тренування: обриває епізод, якщо агент ``patience``
    кроків поспіль не отримав жодної позитивної нагороди.


    У CarRacing позитивна нагорода надходить тільки за нові відвідані
    плитки траси. Якщо її довго немає -- машина з'їхала на траву, крутиться
    на місці або стоїть, і решта епізоду (до 1000 кадрів) дає лише
    штраф -0.1 за кадр та майже нічого не навчає. Обрив позначаємо як
    ``truncated`` (а не ``terminated``), тож у цілі навчання ми все одно
    бутстрепимося з Q(s') -- тобто це не змінює задачу, а лише економить
    симуляцію. Перші ~50 кадрів епізоду (анімація наближення камери)
    також не мають позитивних нагород, тому ``patience`` має бути більшим.
    """


    def __init__(self, env, patience=100):
```

```
        super().__init__(env)
        self.patience = patience
        self.no_progress = 0  # лічильник кроків без позитивної нагороди

    def reset(self, **kwargs):
        self.no_progress = 0
        return self.env.reset(**kwargs)

    def step(self, action):
        obs, reward, terminated, truncated, info = self.env.step(action)
        if reward > 0:
            self.no_progress = 0
        else:
            self.no_progress += 1
        if self.no_progress >= self.patience and not terminated:
            truncated = True
        return obs, reward, terminated, truncated, info


def make_train_env():
    """Тренувальне середовище = незмінне ``make_env()`` + ранній обрив."""
    return NoProgressTruncation(make_env(), patience=100)


def make_vec_env(num_envs, train=True):
    """Кілька екземплярів CarRacing, що крокують паралельно в підпроцесах.

    Режим автоскидання NEXT_STEP (типовий у gymnasium 1.x): після кроку, на
    якому під-середовище завершилось, НАСТУПНИЙ виклик step() повертає
    початкове спостереження нового епізоду (дія на цьому кроці ігнорується).
    """
    fn = make_train_env if train else make_env
    return gym.vector.AsyncVectorEnv(
        [fn for _ in range(num_envs)],
        autoreset_mode=gym.vector.AutoresetMode.NEXT_STEP,
    )


# ---------------------------------------------------------------------------
# Модель
# ---------------------------------------------------------------------------
class QNetwork(nn.Module):
    """Dueling-CNN: стек кадрів -> Q-значення дій.

    Згорткова частина така сама, як у Nature DQN (Mnih et al., 2015):
    3 згорткові шари, вихід 64 x 7 x 7 = 3136 ознак. Далі -- дві окремі
    повнозв'язні гілки (Wang et al., 2016):
      * value-гілка  оцінює V(s) -- наскільки добрий сам стан;
      * advantage-гілка оцінює A(s, a) -- наскільки кожна дія краща за середню.
    Q(s, a) = V(s) + A(s, a) - mean_a A(s, a). Віднімання середнього робить
    розклад однозначним. Перевага: V(s) навчається на кожному переході
    незалежно від того, яку дію було обрано, що прискорює і стабілізує
    навчання, коли багато дій мають близьку цінність (як у CarRacing на
    прямих ділянках).
    """

    def __init__(self, in_channels=FRAME_STACK, a_size=5):
        super().__init__()
        self.a_size = a_size

        # Згорткова частина (ідентична бейзлайну)
        self.conv1 = nn.Conv2d(in_channels, 32, kernel_size=8, stride=4)  # 84 -> 20
        self.conv2 = nn.Conv2d(32, 64, kernel_size=4, stride=2)           # 20 -> 9
        self.conv3 = nn.Conv2d(64, 64, kernel_size=3, stride=1)           # 9 -> 7
```

```
        # Гілка цінності стану V(s)
        self.fc_value = nn.Linear(64 * 7 * 7, 512)
        self.value = nn.Linear(512, 1)
        # Гілка переваги дій A(s, a)
        self.fc_adv = nn.Linear(64 * 7 * 7, 512)
        self.advantage = nn.Linear(512, a_size)


    def forward(self, x):
        # Кадри зберігаються як uint8 [0, 255]; нормалізуємо до [0, 1] вже на пристрої
        x = x.float() / 255.0
        x = F.relu(self.conv1(x))
        x = F.relu(self.conv2(x))
        x = F.relu(self.conv3(x))
        x = x.reshape(x.size(0), -1)


        v = self.value(F.relu(self.fc_value(x)))        # (B, 1)
        a = self.advantage(F.relu(self.fc_adv(x)))      # (B, A)
        return v + a - a.mean(dim=1, keepdim=True)      # (B, A)


    def _device(self):
        # Пристрій беремо з параметрів моделі, щоб act() працював і на CPU
        # (тестовий скрипт), і на MPS/CUDA (тренування).
        return next(self.parameters()).device


    def act(self, state, epsilon=0.0):
        """Epsilon-жадібна дія для одного стану (використовує тестовий скрипт)."""
        if np.random.rand() <= epsilon:
            return np.random.randint(self.a_size)
        state = torch.as_tensor(np.asarray(state), device=self._device()).unsqueeze(0)
        with torch.no_grad():
            q_values = self.forward(state)
        return torch.argmax(q_values, dim=1).item()


    def act_batch(self, states, epsilon=0.0):
        """Epsilon-жадібні дії для батчу станів (по одному на середовище)."""
        states = torch.as_tensor(np.asarray(states), device=self._device())
        with torch.no_grad():
            actions = self.forward(states).argmax(dim=1).cpu().numpy()
        explore = np.random.rand(len(actions)) < epsilon
        actions[explore] = np.random.randint(self.a_size, size=int(explore.sum()))
        return actions



# ---------------------------------------------------------------------------
# Буфер відтворення досвіду з n-кроковими поверненнями
# ---------------------------------------------------------------------------
class NStepReplayBuffer:
    """Кільцевий буфер, що зберігає ПОТІК спостережень кожного середовища.


    Бейзлайн зберігав для кожного переходу і state, і next_state, тобто
    кожен кадр лежав у пам'яті двічі. Тут для кожного з E середовищ є власна
    "стрічка" з C комірок; на кожному кроці векторного середовища в усі
    стрічки дописується рівно одне нове спостереження. Наступний стан
    переходу з комірки j -- це комірка j + 1 тієї ж стрічки, тож дублювання
    немає і в ту саму пам'ять уміщується вдвічі більше переходів.


    Для комірки j зберігаємо дію a_j, нагороду r_j, ознаки terminated_j та
    end_j (епізод закінчився після цього кроку: terminated або truncated), а
    також valid_j -- чи є перехід j -> j+1 справжнім кроком середовища.
    Невалідний перехід виникає на кроці автоскидання: спостереження j --
    останній кадр старого епізоду, j+1 -- перший кадр нового.


    Під час вибірки обчислюється n-крокове повернення
        R = r_j + g r_{j+1} + ... + g^{m-1} r_{j+m-1},
```

```
    де m <= n (менше, якщо епізод завершився раніше), а також коефіцієнт
    бутстрепу g^m (або 0, якщо епізод завершився термінально). Ціль:
        y = R + g^m * Q_target(s_{j+m}, argmax_a Q_online(s_{j+m}, a)).
    """


    def __init__(self, num_envs, capacity_per_env, obs_shape, n_step=3, gamma=0.99):
        self.E = num_envs
        self.C = capacity_per_env
        self.n = n_step
        self.gamma = gamma


        # uint8 -- щоб зменшити пам'ять і трафік CPU -> GPU у 4 рази
        self.obs = np.zeros((num_envs, capacity_per_env, *obs_shape), dtype=np.uint8)
        self.actions = np.zeros((num_envs, capacity_per_env), dtype=np.int64)
        self.rewards = np.zeros((num_envs, capacity_per_env), dtype=np.float32)
        self.terms = np.zeros((num_envs, capacity_per_env), dtype=bool)
        self.ends = np.zeros((num_envs, capacity_per_env), dtype=bool)
        self.valid = np.zeros((num_envs, capacity_per_env), dtype=bool)
        # t -- "абсолютний" індекс поточного (останнього записаного) спостереження;
        # фізична комірка = t % C
        self.t = 0


    def start(self, states):
        """Записати початкові спостереження після env.reset()."""
        self.t = 0
        self.obs[:, 0] = states


    def add(self, actions, rewards, terms, ends, valid, next_states):
        """Додати один крок векторного середовища (по переходу на кожне
середовище)."""
        k = self.t % self.C
        self.actions[:, k] = actions
        self.rewards[:, k] = rewards
        self.terms[:, k] = terms
        self.ends[:, k] = ends
        self.valid[:, k] = valid


        k1 = (self.t + 1) % self.C
        self.obs[:, k1] = next_states
        # Метадані комірки k1 ще не записані (там могли лишитися старі дані)
        self.valid[:, k1] = False
        self.t += 1


    def _window(self):
        """Діапазон абсолютних індексів j, з яких можна почати n-кроковий перехід.


        Потрібні спостереження j..j+n та метадані j..j+n-1, які ще не
        перезаписані: j >= t - C + 1 та j + n <= t.
        """
        lo = max(0, self.t - self.C + 1)
        hi = self.t - self.n
        return lo, hi


    def __len__(self):
        lo, hi = self._window()
        return max(0, hi - lo + 1) * self.E


    def sample(self, batch_size, to_device):
        lo, hi = self._window()


        # Відбір випадкових (середовище, індекс) з відкиданням невалідних
        # переходів (кроків автоскидання). Їх лише ~0.1-1%, тому достатньо
        # одного-двох раундів.
        env_idx, j_idx = [], []
        need = batch_size
```

```
        while need > 0:
            e = np.random.randint(0, self.E, size=2 * need)
            j = np.random.randint(lo, hi + 1, size=2 * need)
            ok = self.valid[e, j % self.C]
            env_idx.append(e[ok][:need])
            j_idx.append(j[ok][:need])
            need -= len(env_idx[-1])
        e = np.concatenate(env_idx)
        j = np.concatenate(j_idx)


        # Векторизоване обчислення n-крокового повернення
        ret = np.zeros(batch_size, dtype=np.float32)
        alive = np.ones(batch_size, dtype=bool)        # епізод ще триває
        boot = np.ones(batch_size, dtype=np.float32)   # 0, якщо епізод завершився
термінально
        disc_boot = np.ones(batch_size, dtype=np.float32)  # g^m для бутстрепу
        nxt = j.copy()                                 # абсолютний індекс s_{j+m}
        disc = 1.0
        for s in range(self.n):
            idx = (j + s) % self.C
            ret += np.where(alive, disc * self.rewards[e, idx],
0.0).astype(np.float32)
            nxt = np.where(alive, j + s + 1, nxt)
            disc_boot = np.where(alive, disc * self.gamma, disc_boot)
            boot = np.where(alive & self.terms[e, idx], 0.0, boot)
            alive = alive & ~self.ends[e, idx]
            disc *= self.gamma


        states = torch.from_numpy(self.obs[e, j % self.C]).to(to_device)
        next_states = torch.from_numpy(self.obs[e, nxt % self.C]).to(to_device)
        actions = torch.from_numpy(self.actions[e, j %
self.C]).to(to_device).unsqueeze(1)
        returns = torch.from_numpy(ret).to(to_device).unsqueeze(1)
        bootstrap = torch.from_numpy((boot *
disc_boot).astype(np.float32)).to(to_device).unsqueeze(1)
        return states, actions, returns, next_states, bootstrap



# ----------------------------------------------------------------------------
# Розклади гіперпараметрів
# ----------------------------------------------------------------------------
def piecewise_linear(progress, points):
    """Кусково-лінійна інтерполяція: points = [(x0, y0), (x1, y1), ...]."""
    if progress <= points[0][0]:
        return points[0][1]
    for (x0, y0), (x1, y1) in zip(points[:-1], points[1:]):
        if progress <= x1:
            return y0 + (progress - x0) / (x1 - x0) * (y1 - y0)
    return points[-1][1]



# Epsilon: швидко 1.0 -> 0.1 за перші 20% кроків (активне дослідження),
# далі повільно 0.1 -> 0.01 до 60%, і 0.01 до кінця (майже жадібна політика).
EPSILON_SCHEDULE = [(0.0, 1.0), (0.2, 0.1), (0.6, 0.01)]



# ----------------------------------------------------------------------------
# Агент
# ----------------------------------------------------------------------------
class DQNAgent:
    def __init__(self, action_dim, obs_shape, num_envs, lr=1e-4, lr_min=1e-5,
                 gamma=0.99, n_step=3, buffer_per_env=15000, batch_size=64,
                 tau=0.0025, total_updates=1):
        self.action_dim = action_dim
        self.gamma = gamma
        self.batch_size = batch_size
```

```
        self.tau = tau
        self.lr = lr
        self.lr_min = lr_min
        self.total_updates = max(1, total_updates)
        self.epsilon = 1.0

        # batch_size = 64 і 2 оновлення на крок векторного середовища: та сама
        # кількість семплів, що й 1 х 128, але вдвічі більше градієнтних кроків
        # (на MPS батч 64 рахується ~1.6 раза швидше за 128).
        self.model = QNetwork(a_size=action_dim).to(train_device)
        self.target_model = QNetwork(a_size=action_dim).to(train_device)
        self.target_model.load_state_dict(self.model.state_dict())
        self.target_model.eval()
        for p in self.target_model.parameters():
            p.requires_grad_(False)
        self.online_params = list(self.model.parameters())
        self.target_params = list(self.target_model.parameters())

        # eps=1.5e-4 -- стандартне значення Adam для DQN (Rainbow, Dopamine):
        # робить крок менш чутливим до дуже малих оцінок другого моменту
        self.optimizer = optim.Adam(self.model.parameters(), lr=lr, eps=1.5e-4)
        self.memory = NStepReplayBuffer(num_envs, buffer_per_env, obs_shape,
                                        n_step=n_step, gamma=gamma)
        self.learn_steps = 0
        self.last_loss = float("nan")
        self.last_q = float("nan")

    def _update_lr(self):
        """Косинусне згасання learning rate від lr до lr_min за все навчання.

        На початку великий крок швидко навчає, наприкінці малий крок
        "заспокоює" ваги, і політика менше коливається між чекпоінтами.
        """
        progress = min(1.0, self.learn_steps / self.total_updates)
        lr = self.lr_min + 0.5 * (self.lr - self.lr_min) * (1 + math.cos(math.pi *
progress))
        for g in self.optimizer.param_groups:
            g["lr"] = lr
        return lr

    def replay(self):
        """Один крок градієнтного спуску на мінібатчі з буфера."""
        if len(self.memory) < self.batch_size:
            return

        states, actions, returns, next_states, bootstrap = self.memory.sample(
            self.batch_size, train_device)

        # Q(s, a) поточної мережі для дій, які реально виконувались
        q_values = self.model(states).gather(1, actions)

        with torch.no_grad():
            # Double DQN: argmax обирає ОНЛАЙН-мережа ...
            next_actions = self.model(next_states).argmax(dim=1, keepdim=True)
            # ... а значення цієї дії бере ЦІЛЬОВА мережа
            next_q = self.target_model(next_states).gather(1, next_actions)
            target = returns + bootstrap * next_q

        # Huber-втрата: квадратична для малих помилок, лінійна для великих --
        # стійка до рідкісних великих TD-помилок (наприклад, -100 за виїзд за поле)
        loss = F.smooth_l1_loss(q_values, target)

        self._update_lr()
        self.optimizer.zero_grad(set_to_none=True)
        loss.backward()
```

```
        nn.utils.clip_grad_norm_(self.model.parameters(), 10.0)
        self.optimizer.step()
        self.learn_steps += 1

        # М'яке оновлення цільової мережі: theta' <- tau*theta + (1-tau)*theta'.
        # Ціль змінюється плавно на кожному кроці, без "стрибків" раз на N кроків.
        # Одним "foreach"-викликом для всіх тензорів -- менше запусків ядер на GPU.
        # tau = 0.0025 -> стала часу ~400 оновлень (близько до жорсткого
        # оновлення раз на 1000 кроків у бейзлайні, але без стрибків).
        with torch.no_grad():
            torch._foreach_lerp_(self.target_params, self.online_params, self.tau)

        # Статистика для логів (рідко, бо .item() синхронізує GPU)
        if self.learn_steps % 500 == 0:
            self.last_loss = loss.item()
            self.last_q = q_values.mean().item()


# ---------------------------------------------------------------------------
# Оцінювання жадібної політики (для вибору чекпоінта)
# ---------------------------------------------------------------------------
def evaluate_policy_vec(model, eval_env, num_episodes=16):
    """Прогнати ``num_episodes`` повних епізодів жадібної політики (epsilon = 0)
    на НЕЗМІННОМУ середовищі ``make_env`` (без раннього обриву).

    Використовуємо векторне середовище: кожне з E під-середовищ проходить
    ceil(num_episodes / E) епізодів. Повертає масив нагород за епізоди.
    """
    num_envs = eval_env.num_envs
    per_env = math.ceil(num_episodes / num_envs)
    was_training = model.training
    model.eval()

    states, _ = eval_env.reset()
    ep_ret = np.zeros(num_envs)
    finished = np.zeros(num_envs, dtype=int)
    prev_done = np.zeros(num_envs, dtype=bool)
    results = []
    while (finished < per_env).any():
        actions = model.act_batch(states, epsilon=0.0)
        states, rewards, terms, truncs, _ = eval_env.step(actions)
        dones = np.logical_or(terms, truncs)
        for i in range(num_envs):
            if prev_done[i]:
                continue  # крок автоскидання -- не частина епізоду
            ep_ret[i] += rewards[i]
            if dones[i]:
                if finished[i] < per_env:
                    results.append(ep_ret[i])
                finished[i] += 1
                ep_ret[i] = 0.0
        prev_done = dones

    if was_training:
        model.train()
    return np.array(results[:num_episodes])


# ---------------------------------------------------------------------------
# Тренування
# ---------------------------------------------------------------------------
def train(num_envs=8, total_steps=1_500_000, updates_per_step=2,
learning_starts=20_000,
          eval_every=50_000, eval_episodes=16, seed=42, results_dir=RESULTS_DIR):
    """Тренування з кількома паралельними середовищами CarRacing.
```

```
    Бюджет задається кількістю кроків середовища (кадрів), а не епізодів:
    через ранній обрив тренувальні епізоди мають різну довжину. Для
    порівняння: 1000 епізодів бейзлайну -- це ~1 000 000 кадрів.


    На кожній ітерації:
      1. агент обирає дії для всіх E середовищ (epsilon-жадібно);
      2. дії надсилаються в підпроцеси (step_async) і, ПОКИ середовища
         симулюють крок, виконується ``updates_per_step`` оновлень мережі на GPU;
      3. результати кроку (step_wait) записуються в буфер.
    """
    os.makedirs(results_dir, exist_ok=True)
    np.random.seed(seed)
    torch.manual_seed(seed)


    env = make_vec_env(num_envs, train=True)
    eval_env = make_vec_env(num_envs, train=False)  # незмінне середовище для оцінки


    total_vec_steps = total_steps // num_envs
    total_updates = max(1, (total_vec_steps - learning_starts // num_envs) *
updates_per_step)
    agent = DQNAgent(env.single_action_space.n, env.single_observation_space.shape,
                     num_envs=num_envs, total_updates=total_updates)
    print(f"train device: {train_device}, envs: {num_envs}, total steps:
{total_steps}, "
          f"updates: {total_updates}")


    # Логи для графіків у звіті
    train_log = open(os.path.join(results_dir, "train_log.csv"), "w", newline="")
    train_writer = csv.writer(train_log)
    train_writer.writerow(["episode", "env_steps", "score", "length", "epsilon",
"truncated_early",
                           "wall_time"])
    eval_log = open(os.path.join(results_dir, "eval_log.csv"), "w", newline="")
    eval_writer = csv.writer(eval_log)
    eval_writer.writerow(["env_steps", "episodes", "mean", "std", "min", "max", "lr",
"loss",
                          "mean_q", "wall_time"])


    scores_deque = deque(maxlen=100)
    best_eval = -np.inf
    start_time = time.time()


    states, _ = env.reset(seed=seed)
    agent.memory.start(states)
    prev_done = np.zeros(num_envs, dtype=bool)
    episode_rewards = np.zeros(num_envs)
    episode_lengths = np.zeros(num_envs, dtype=int)
    completed = 0
    env_steps = 0
    next_eval = eval_every


    for vec_step in range(total_vec_steps):
        agent.epsilon = piecewise_linear(env_steps / total_steps, EPSILON_SCHEDULE)
        actions = agent.model.act_batch(states, epsilon=agent.epsilon)


        # Середовища крокують у підпроцесах паралельно з навчанням мережі
        env.step_async(actions)
        if env_steps >= learning_starts:
            for _ in range(updates_per_step):
                agent.replay()
        next_states, rewards, terms, truncs, _ = env.step_wait()
        dones = np.logical_or(terms, truncs)


        # Перехід валідний, якщо попередній крок не завершив епізод
```

```
        # (інакше це крок автоскидання: next_states -- початок нового епізоду)
        agent.memory.add(actions, rewards, terms, dones, ~prev_done, next_states)


        for i in range(num_envs):
            if prev_done[i]:
                continue
            env_steps += 1
            episode_rewards[i] += rewards[i]
            episode_lengths[i] += 1
            if dones[i]:
                score = episode_rewards[i]
                completed += 1
                scores_deque.append(score)
                early = bool(truncs[i] and episode_lengths[i] < 1000)
                train_writer.writerow([completed, env_steps, f"{score:.2f}",
episode_lengths[i],
                                       f"{agent.epsilon:.4f}", int(early),
                                       f"{time.time() - start_time:.1f}"])
                if completed % 10 == 0:
                    train_log.flush()
                    print(f"Episode {completed}\tSteps {env_steps}\tScore {score:.1f}
\t"
                          f"Avg100 {np.mean(scores_deque):.1f}\tEps
{agent.epsilon:.3f}\t"
                          f"Loss {agent.last_loss:.3f}\tQ {agent.last_q:.1f}\t"
                          f"t {(time.time() - start_time) / 60:.1f} min")
                episode_rewards[i] = 0
                episode_lengths[i] = 0


        states = next_states
        prev_done = dones


        # Періодична жадібна оцінка на справжньому (тестовому) середовищі
        if env_steps >= next_eval or vec_step == total_vec_steps - 1:
            next_eval += eval_every
            res = evaluate_policy_vec(agent.model, eval_env, eval_episodes)
            lr = agent.optimizer.param_groups[0]["lr"]
            eval_writer.writerow([env_steps, len(res), f"{res.mean():.2f}",
f"{res.std():.2f}",
                                  f"{res.min():.2f}", f"{res.max():.2f}", f"{lr:.2e}",
                                  f"{agent.last_loss:.4f}", f"{agent.last_q:.2f}",
                                  f"{time.time() - start_time:.1f}"])
            eval_log.flush()
            msg = (f"  [EVAL] steps {env_steps}: mean {res.mean():.1f} std
{res.std():.1f} "
                   f"min {res.min():.1f} max {res.max():.1f}")
            if res.mean() > best_eval:
                best_eval = res.mean()
                torch.save({k: v.cpu() for k, v in agent.model.state_dict().items()},
                           CHECKPOINT_BEST)
                msg += f"  -> new best, saved {CHECKPOINT_BEST}"
            print(msg)


    torch.save({k: v.cpu() for k, v in agent.model.state_dict().items()},
CHECKPOINT_LAST)
    train_log.close()
    eval_log.close()
    env.close()
    eval_env.close()
    print(f"Done in {(time.time() - start_time) / 60:.1f} min. Best eval mean
{best_eval:.1f}")



# ---------------------------------------------------------------------------
# Перегляд / запис відео (як у бейзлайні)
# ---------------------------------------------------------------------------
def test(env, policy, num_episodes=5):
```

```
    total_reward = 0
    for _ in range(num_episodes):
        state, _ = env.reset()
        done = False
        while not done:
            action = policy.act(state)
            state, reward, terminated, truncated, _ = env.step(action)
            done = terminated or truncated
            total_reward += reward
    avg = total_reward / num_episodes
    print(f"Total Reward: {avg:.1f}")
    return avg



def render(checkpoint=CHECKPOINT_BEST, num_episodes=3, video_folder="videos"):
    """Записати відео гри натренованого агента у MP4 (працює без дисплея)."""
    env = make_env(render_mode="rgb_array")
    env = RecordVideo(
        env,
        video_folder=video_folder,
        name_prefix="car_racing",
        episode_trigger=lambda e: True,
    )
    policy = QNetwork(a_size=env.action_space.n).to(device)
    policy.load_state_dict(torch.load(checkpoint, map_location=device))
    policy.eval()
    test(env, policy, num_episodes=num_episodes)
    env.close()
    print(f"Saved videos to {video_folder}/")



if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Improved DQN for CarRacing-v3")
    parser.add_argument("--num-envs", type=int, default=8)
    parser.add_argument("--total-steps", type=int, default=1_500_000)
    parser.add_argument("--updates-per-step", type=int, default=2)
    parser.add_argument("--eval-every", type=int, default=50_000)
    parser.add_argument("--eval-episodes", type=int, default=16)
    parser.add_argument("--seed", type=int, default=42)
    parser.add_argument("--results-dir", default=RESULTS_DIR)
    parser.add_argument("--render", action="store_true", help="лише записати відео з
чекпоінта")
    args = parser.parse_args()


    if args.render:
        render()
    else:
        train(num_envs=args.num_envs, total_steps=args.total_steps,
              updates_per_step=args.updates_per_step, eval_every=args.eval_every,
              eval_episodes=args.eval_episodes, seed=args.seed,
results_dir=args.results_dir)
```